b093b1dd-4d53-4562-b1b8-39c925e951a3

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'microsoft/vscode' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/vscode/vscode (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-17T00:11:44.799+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Bump esbuild, @vitest/coverage-v8, tsx, vite, vite-plugin-wasm and vitest in /extensions/copilot
</pr_title>

<pr_description>
Bumps esbuild to 0.28.1 and updates ancestor dependencies esbuild, @vitest/coverage-v8, tsx, vite, vite-plugin-wasm and <a href="https://github.com/vitest-dev/vitest/tree/HEAD/packages/

vitest">vitest. These dependencies need to be updated together.

Updates `esbuild` from 0.27.2 to 0.28.1

Updates `@vitest/coverage-v8` from 3.2.4 to 4.1.8

Updates `tsx` from 4.20.3 to 4.22.4

Updates `vite` from 7.3.2 to 8.0.16

Additional fixes for chat-lib CI failures:

- Updated `chat-lib/package.json` to use the bumped versions (`tsx: ^4.22.4`, `vitest: ^4.1.8`) so the chat-lib is consistent with the main extension.
- Regenerated `chat-lib/package-lock.json` to include new vitest 4.x transitive dependencies (`rolldown`, `lightningcss`, `@standard-schema/spec`, `vite 8.x`, etc.) that were missing and caused `npm ci` to fail in the chat-lib tests workflow.
- Fixed a bug in `extractChatLib.ts` where the lock file sync logic only updated *existing* entries but skipped adding brand-new packages introduced by version bumps, causing the chat-lib lock file to be incomplete after syncing.
- Fixed a second bug in `extractChatLib.ts` where `jsx-dev-runtime` was not being extracted alongside `jsx-runtime` when processing `@jsxImportSource` pragmas in `.tsx` files, causing chat-lib tests to fail with `Cannot find module '…/jsx-dev-runtime'`.
- Fixed vitest 4.x JSX runtime resolution: removed trailing slashes from `@jsxImportSource` pragmas in 21 source files — vitest 4.x (vite 8.x) no longer normalizes the path before appending `/jsx-dev-runtime`, causing a double-slash path (`jsx-runtime//jsx-dev-runtime`) that fails module resolution.
- Added `jsx-dev-runtime.ts` to the custom `jsx-runtime` directory, re-exporting `jsx` as `jsxDEV` and `Fragment` — required by vitest 4.x dev mode which explicitly resolves `jsx-dev-runtime` for JSX transforms.
- Fixed vitest 4.x `Mock` type compatibility: replaced

`ReturnType<typeof vi.fn>` with `Mock` (imported from vitest) in 4 test files (`genAiEvents.spec.ts`, `genAiMetrics.spec.ts`, `copilotCLITerminalIntegration.spec.ts`, `terminalCommand.spec.ts`) — in vitest 4.x `ReturnType<typeof vi.fn>` resolves to `Mock<Constructable | Procedure>` which is not assignable to specific function types, whereas `Mock` (= `Mock<Procedure>`) has `any`-typed call signatures that are compatible with all function types.
- Removed `vite-plugin-top-level-await` from `vite.config.ts` and from `package.json` devDependencies — Vite 8 replaced rollup with rolldown, causing this plugin's direct `require("rollup")` call to fail at startup with `Cannot find module 'rollup'`, aborting all tests before they could run. Top-level await is natively supported in Vite 8/Rolldown, so the plugin is no longer needed.
- Added `oxc: { jsx: { development: false } }` to `vite.config.ts` — Vite 8 uses OXC for transforms and defaults to `development: true`, which injects `__self: this` and `__source: {...}` debug props into classic JSX calls. `@vscode/prompt-tsx` stores all props as-is, so `this` (containing full snapshot text) leaked into `JSON.stringify` output, breaking `not.toContain` assertions in `inlineChat2Prompt.spec.tsx`. Disabling OXC development mode prevents these debug props from being added.
- Added `vi.restoreAllMocks()` to the `afterEach` in `openAIEndpoint.spec.ts` — in vitest 4.x, `vi.spyOn` on an already-spied prototype method (not yet restored) reuses the existing spy, causing call counts to accumulate across tests. The preceding test's spy call count carried over, causing `toHaveBeenCalledOnce()` to fail with count 2.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

ChatLib. Also, why was package-lock regenerated from scratch? Can you undo your changes to it and update only based on changes to package.json?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Revert improvements to extractChatLib. Also, why was package-lock regenerated from scratch? Can you undo your

changes to it and update only based on changes to package.json?

Done in the latest commit.

`extractChatLib.ts` improvements reverted — removed the "sync ALL shared packages" logic and restored the original targeted sync (only updated packages + their transitive deps, with the fix to add new entries not previously in the chat-lib lock).

Package-lock regeneration — the scratch regeneration happened because `npm install --package-lock-only` wasn't run at the right point in the history; instead we deleted and re-created from nothing. Now fixed: restored both `extensions/copilot/package-lock.json` and `extensions/copilot/chat-lib/package-lock.json` to their original pre-branch state and ran `npm install --package-lock-only --ignore-scripts` so only the packages affected by the `package.json` version bumps are updated, keeping everything else locked to the original versions.
</comment_old>

<comment_new>
<comment_id>4724682739</comment_id>
<author>@dmitrivMS</author>
@copilot there is still a merge conflict - can you undo changes to package-lock.json, merge, then update it from package.json with npm install?
</comment_new>

</pr_comments>

</comments>

The last **22** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.

* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prompt>

@copilot there is still a merge conflict - can you undo changes to package-lock.json, merge, then update it from package.json with npm install?

<prev_assistant>

Task completed.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'microsoft/vscode' repository.

<repository_context>

I've cloned the repository in the directory /home/runner/work/vscode/vscode (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-17T00:28:14.632+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Bump esbuild, @vitest/coverage-v8, tsx, vite, vite-plugin-wasm and vitest in /extensions/copilot
</pr_title>

<pr_description>
Bumps esbuild to 0.28.1 and updates ancestor dependencies esbuild, @vitest/coverage-v8, tsx, vite, vite-plugin-wasm and vitest. These dependencies need to be updated together.

Updates `esbuild` from 0.27.2 to 0.28.1

Updates `@vitest/coverage-v8` from 3.2.4 to 4.1.8

Updates `tsx` from 4.20.3 to 4.22.4

Updates `vite` from 7.3.2 to 8.0.16

Additional fixes for chat-lib CI failures:

- Updated `chat-lib/package.json` to use the bumped versions (`tsx: ^4.22.4`, `vitest: ^4.1.8`) so the chat-lib is consistent with the main extension.
- Regenerated `chat-lib/package-lock.json` to include new vitest 4.x transitive dependencies (`rolldown`, `lightningcss`, `@standard-schema/spec`, `vite 8.x`, etc.) that were missing and caused `npm ci` to fail in the chat-lib tests workflow.
- Fixed a bug in `extractChatLib.ts` where the lock file sync logic only updated *existing* entries but skipped adding brand-new packages introduced by version bumps, causing the chat-lib lock file to be incomplete after syncing.
- Fixed a second bug in `extractChatLib.ts` where `jsx-dev-runtime` was not being extracted alongside `jsx-runtime` when processing `@jsxImportSource` pragmas in `.tsx` files, causing chat-lib tests to fail with `Cannot find module '…/jsx-dev-runtime'`.
- Fixed vitest 4.x JSX runtime resolution: removed trailing slashes from `@jsxImportSource` pragmas in 21 source files — vitest 4.x (vite 8.x) no longer normalizes the path before appending `/jsx-dev-runtime`, causing a double-slash path (`jsx-runtime//jsx-dev-runtime`) that fails module resolution.
- Added `jsx-dev-runtime.ts` to the custom `jsx-runtime` directory, re-exporting `jsx` as `jsxDEV` and `Fragment` — required by vitest 4.x dev mode which explicitly resolves `jsx-dev-runtime` for JSX transforms.
- Fixed vitest 4.x `Mock` type compatibility: replaced `ReturnType<typeof vi.fn>` with `Mock` (imported from vitest) in 4 test files (`genAiEvents.spec.ts`, `genAiMetrics.spec.ts`, `copilotCLITerminalIntegration.spec.ts`, `terminalCommand.spec.ts`) — in vitest 4.x `ReturnType<typeof vi.fn>` resolves to `Mock<Constructable | Procedure>` which is not assignable to specific function types, whereas `Mock` (= `Mock<Procedure>`) has `any`-typed call signatures that are compatible with all function types.
- Removed `vite-plugin-top-level-await` from `vite.config.ts` and from `package.json` devDependencies — Vite 8 replaced rollup with rolldown, causing this plugin's direct `require("rollup")` call to fail at startup with `Cannot find module 'rollup'`, aborting all tests before they could run. Top-level await is natively supported in Vite 8/Rolldown, so the plugin is no longer needed.
- Added `oxc: { jsx: { development: false } }` to `vite.config.ts` — Vite 8 uses OXC for transforms and defaults to `development: true`, which injects `__self: this` and

`__source: {...}` debug props into classic JSX calls. `@vscode/prompt-tsx` stores all props as-is, so `this` (containing full snapshot text) leaked into `JSON.stringify` output, breaking `not.toContain` assertions in `inlineChat2Prompt.spec.tsx`. Disabling OXC development mode prevents these debug props from being added.
- Added `vi.restoreAllMocks()` to the `afterEach` in `openAIEndpoint.spec.ts` — in vitest 4.x, `vi.spyOn` on an already-spied prototype method (not yet restored) reuses the existing spy, causing call counts to accumulate across tests. The preceding test's spy call count carried over, causing `toHaveBeenCalledOnce()` to fail with count 2.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

s module resolution. Removed trailing slashes from all affected files.

2. **Missing `jsx-dev-runtime` module** — vitest 4.x dev mode requires a `jsx-dev-runtime` module exporting `jsxDEV`. Created `jsx-dev-runtime.ts` in the `jsx-runtime` directory that re-exports `jsx` as `jsxDEV` and `Fragment` from `jsx-runtime.ts`.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot still build break
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot still build break

Fixed in the latest commit. There were two remaining failures:

1. **chat-lib tests** — `extractChatLib.ts` was resolving `jsx-runtime` for `@jsxImportSource` pragmas but not `jsx-dev-runtime`, so the `jsx-dev-runtime.ts` file added in the previous commit was never extracted into chat-lib. Added a second `resolveImportPath` call for `jsx-dev-runtime` in the same loop.

2. **Copilot - Test (Windows/Linux)** — vitest 4.x changed `ReturnType<typeof vi.fn>` to resolve to `Mock<Constructable | Procedure>` (using the constraint union) instead of `Mock<Procedure>`. This caused type errors in 4 test files where that type was used in interface declarations and function return types. Fixed by importing `Mock` from vitest and using it directly in place of `ReturnType<typeof vi.fn>`.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot Still CI failures
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Still CI failures

Fixed in the latest commit. The remaining failure was caused by `vite-plugin-top-level-await` requiring `rollup` directly — Vite 8 replaced rollup with rolldown, so this plugin fails on startup with `Cannot find module 'rollup'`. Since Vite 8/Rolldown natively supports top-level await, the plugin is no longer needed. Removed the import and usage from `vite.config.ts`.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
> > @copilot Still CI failures
>
> Fixed in the latest commit. The remaining failure was caused by `vite-plugin-top-level-await` requiring `rollup` directly — Vite 8 replaced rollup with rolldown, so this plugin fails on startup with `Cannot find module 'rollup'`. Since Vite 8/Rolldown natively supports top-level await, the plugin is no longer needed. Removed the import and usage from `vite.config.ts`.

@copilot Then perhaps we should remove the dependency on vite-plugin-top-level-await?
</comment_old>

<comment_old>
<author>@copilot</author>

> > > @copilot Still CI failures
> >
> > Fixed in the latest commit. The remaining failure was caused by `vite-plugin-top-level-await` requiring `rollup` directly — Vite 8 replaced rollup with rolldown...

Done — removed `vite-plugin-top-level-await` from `package.json` devDependencies and updated `package-lock.json` accordingly. The import was already removed from `vite.config.ts` in a previous commit.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot Investigate and fix CI failures
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Investigate and fix CI failures

Fixed in the latest commit. Two remaining failures:

1. **`inlineChat2Prompt.spec.tsx`** (5 tests) — Vite 8 uses OXC for transforms and defaults to `development: true`, which injects `__self: this` and `__source: {...}` as extra props on classic JSX calls (old Babel behavior). `@vscode/prompt-tsx` stores all props as-is, so `this` (containing the full snapshot text) ended up in `JSON.stringify(rendered)`, breaking the `not.toContain` assertions. Fixed by adding `oxc: { jsx: { development: false } }` to `vite.config.ts`.

2. **`openAIEndpoint.spec.ts`** (1 test) — In vitest 4.x, `vi.spyOn` on an already-spied prototype method (not yet restored) reuses the existing spy, causing call counts to accumulate across tests. The preceding test's one call to `ChatEndpoint.prototype.makeChatRequest2` carried over, making the count 2 in the next test. Fixed by adding `vi.restoreAllMocks()` to the `afterEach`.

</comment_old>

<comment_old>
<author>@rzhao271</author>
Because this is a large change, could you move the PR to 1.126.0 and also run a custom Insiders build for this branch on Azure DevOps?
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
> Because this is a large change, could you move the PR to 1.126.0 and also run a custom Insiders build for this branch on Azure DevOps?

Will do. I guess I just got carried away trying to help it pass the CI :-)
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
Started custom build: https://dev.azure.com/monacotools/Monaco/_build?definitionId=111

</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@rzhao271 I think this is ready to go.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot Merge everything except package-lock.json and regenerate the lock files.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Merge everything except package-lock.json and regenerate the lock files.

Done. Deleted both `extensions/copilot/package-lock.json` and `extensions/copilot/chat-lib/package-lock.json` and regenerated them from scratch with `npm install --package-lock-only --ignore-scripts`. All other code changes (vite.config.ts,

extractChatLib.ts, jsx-runtime fixes, test fixes, etc.) are preserved.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot merge main, resolve merge conflicts and regenerate package-lock.json files
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot merge main, resolve merge conflicts and regenerate package-lock.json files

Done. Merged `origin/main` into the branch, resolved the `package-lock.json` conflict by regenerating both `extensions/copilot/package-lock.json` and `extensions/copilot/chat-lib/package-lock.json` from scratch after the merge.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot address CI failures
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot address CI failures

Fixed in the latest commit. The chat-lib lock file (regenerated from scratch after the main merge) had picked up newer package versions within the semver ranges that broke compilation:

1. **`vscode-languageserver-protocol@3.18.0`** — version 3.18 switched to an `exports` map, which TypeScript's legacy `node` moduleResolution (used when `module: "commonjs"` without an explicit `moduleResolution`) doesn't support. The error was `Consider updating to 'node16', 'nodenext', or 'bundler'`. Pinned back to `3.17.5` (along with its transitive deps `vscode-jsonrpc@8.2.0` and `vscode-languageserver-types@3.17.5`).

2. **`openai@6.43.0`** — this version changed the image detail type from `"low" | "high" | "original"` to `"low" | "high" | "auto"`, breaking `responsesApi.ts`. Pinned back to `6.7.0`.

Also improved `extractChatLib.ts` to prevent this from recurring on future lock regenerations: it now syncs all shared packages from the copilot extension lock (not just explicitly updated ones), skipping only packages where the copilot extension has a nested higher-versioned install (which would indicate the top-level entry is a legacy pin from a conflicting direct dependency, not the effective version for shared packages).
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot Revert improvements to extractChatLib. Also, why was package-lock regenerated from scratch? Can you undo your changes to it and update only based on changes to package.json?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Revert improvements to extractChatLib. Also, why was package-lock regenerated from scratch? Can you undo your changes to it and update only based on changes to package.json?

Done in the latest commit.

`extractChatLib.ts` improvements reverted — removed the "sync ALL shared packages" logic and restored the original targeted sync (only updated packages + their transitive deps, with the fix to add new entries not previously in the chat-lib lock).

Package-lock regeneration — the scratch regeneration happened because `npm install --package-lock-only` wasn't run at the right point in the history; instead we deleted and re-created from nothing. Now fixed: restored both `extensions/copilot/package-lock.json` and `extensions/copilot/chat-lib/package-lock.json` to their original pre-branch state and ran `npm install --package-lock-only --ignore-scripts` so only the packages affected by the `package.json` version bumps are updated, keeping everything else locked to the original versions.
</comment_old>

<comment_old>
<author>@dmitrivMS</author>
@copilot there is still a merge conflict - can you undo changes

to package-lock.json, merge, then update it from package.json with npm install?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot there is still a merge conflict - can you undo changes to package-lock.json, merge, then update it from package.json with npm install?

Done. Reset both lock files to the `main` version, merged `origin/main` (clean, no conflicts), then ran `npm install --package-lock-only --ignore-scripts` to update them based on the `package.json` changes. Both lock files now have the correct bumped versions (vitest 4.1.9, vite 8.0.16, tsx 4.22.4, esbuild 0.28.1).
</comment_old>

<comment_new>
<comment_id>4724763998</comment_id>
<author>@dmitrivMS</author>
@copilot List packages for main and chat package.json whose versions changed here, but only PRODUCTION packages (omit dev-only dependencies).
</comment_new>

</pr_comments>

</comments>

The last **25** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:

```
    - If it is a request for changes, a question on the code, or
a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```

<prompt>

```
@copilot List packages for main and chat package.json whose versions
changed here, but only PRODUCTION packages (omit dev-only
dependencies).
```